

Model Selection & Application Architecture

PHASE OVERVIEW

Objective

Choose the AI approach and define a modular structure that supports the complete FitBuddy journey.

Included work

AI model choice

Use Google Gemini through the google-genai SDK. Keep model names configurable; the documented default is gemini-2.5-flash.

System blueprint

Connect a Jinja2 user interface to FastAPI routes, Pydantic validation, the AI layer, and SQLite persistence.

Development setup

Prepare Python 3.10+, a virtual environment, dependencies from requirements.txt, and environment configuration.

PHASE OUTPUT

This phase produces a working, reviewable foundation for the next project phase.

Model Selection & Application Architecture

PROJECT PLAN | IMPLEMENTATION & ACCEPTANCE

Architecture decisions & setup

Component map

app/main.py starts FastAPI and mounts static assets; routes.py handles pages and APIs; schemas.py validates payloads; database.py manages SQLite/SQLAlchemy; app/ai/ contains generation logic; templates/ and static/ provide the interface.

Configuration

Read Gemini model settings and API key from environment configuration. Keep demo mode available so local development can run without a live AI key.

Build sequence

Create the virtual environment, install requirements, confirm the app modules import, then start the local server before implementing the user-facing flow.

DELIVERABLES

- Document the selected model and configuration
- Establish module boundaries and request/response contracts
- Provide repeatable local setup instructions

ACCEPTANCE CHECKS

- Dependencies install in the isolated environment
- Architecture maps cleanly to project folders
- AI credentials and demo mode are configurable

Core Functionality & FastAPI Backend

PROJECT PLAN | SCOPE & OBJECTIVES

PHASE OVERVIEW

Objective

Implement the complete profile-to-plan workflow, including AI guidance, feedback, and persistence.

Included work

Collect a fitness profile

Accept username, user ID, age, weight, goal, and intensity; validate values with Pydantic and reject duplicate IDs.

Generate personalized guidance

Create a seven-day workout plan and nutrition tip using the configured AI generation layer.

Capture feedback

Save user feedback and generate an updated plan using the existing profile and original plan.

Persist user data

Store profile, generated workout, nutrition guidance, feedback, and updated plan in SQLite through SQLAlchemy.

PHASE OUTPUT

This phase produces a working, reviewable foundation for the next project phase.

Core Functionality & FastAPI Backend

PROJECT PLAN | IMPLEMENTATION & ACCEPTANCE

Backend routes & data flow

Web routes

POST /generate-workout creates a profile and renders results. POST /submit-feedback updates a saved plan. GET /view-all-users presents the admin view.

JSON API routes

POST /api/generate-workout and POST /api/submit-feedback support programmatic use; GET /api/users exposes user records. FastAPI also provides /docs.

Reliability path

Validate inputs before writing; persist generated results; handle generation errors and return useful HTTP errors. Keep web and API behavior aligned.

Data lifecycle

Profile - validation - AI workout and nutrition - database - result response - feedback - updated plan.

DELIVERABLES

- Validated profile and feedback schemas
- Web and JSON endpoints for generation and feedback
- Persistent records for original and updated plans

ACCEPTANCE CHECKS

- Invalid or duplicate profiles are rejected
- Successful generation returns workout and nutrition guidance
- Feedback updates the plan and persists the change

Application Entry Point & Integration

PROJECT PLAN | SCOPE & OBJECTIVES

PHASE OVERVIEW

Objective

Bring the application modules together in a runnable FastAPI service with database initialization and a health check.

Included work

Create the app

Instantiate FastAPI with the FitBuddy application title and description.

Connect modules

Mount the static directory, include the routes router, and keep paths rooted from the project folder.

Initialize storage

Create database tables at application startup so the persistence layer is ready.

Expose service status

Provide GET /health with a lightweight healthy response for a local operational check.

PHASE OUTPUT

This phase produces a working, reviewable foundation for the next project phase.

Application Entry Point & Integration

PROJECT PLAN | IMPLEMENTATION & ACCEPTANCE

Run, connect & verify

Startup path

Launch with `uvicorn app.main:app --reload` from the project directory. The app entry point loads configuration, routes, static resources, and database startup behavior.

Integration checks

Confirm the home page and route handlers load; confirm templates and static CSS resolve; confirm the database can be initialized.

Operational endpoints

Open `/health` to inspect service status and `/docs` to inspect the generated OpenAPI interface.

Configuration handoff

Local settings are loaded from `.env`. Keep the Gemini key and demo-mode choice consistent with `app/config.py` and the selected run mode.

DELIVERABLES

- Runnable FastAPI entry point
- Startup database initialization
- Mounted static assets and registered routes
- Health and interactive API documentation endpoints

ACCEPTANCE CHECKS

- Server starts without import or configuration errors
- GET `/health` returns a healthy status
- Home page, static assets, and API documentation respond

Frontend & Dynamic Templates

PROJECT PLAN | SCOPE & OBJECTIVES

PHASE OVERVIEW

Objective

Give users a clear path to enter their profile, review their personalized plan, and submit feedback.

Included work

Profile form

Use templates/index.html to collect the required fitness details and send them to the generation route.

Result experience

Use templates/result.html to show the user profile, workout plan, nutrition tip, errors, and updated plan when available.

Feedback loop

Let users provide feedback from the result flow and display the refreshed plan returned by the backend.

Administration view

Use templates/all_users.html for the protected user listing; static/style.css supplies the responsive visual design.

PHASE OUTPUT

This phase produces a working, reviewable foundation for the next project phase.

Frontend & Dynamic Templates

PROJECT PLAN | IMPLEMENTATION & ACCEPTANCE

Template behavior & experience

Dynamic rendering

Pass request and result values from FastAPI into Jinja2 templates, including user, workout_plan, nutrition_tip, updated_plan, and error state.

User journey

Home form - generated workout and nutrition result - feedback submission - updated plan. Ensure each state has clear headings and next actions.

Responsive styling

Apply shared CSS for readable layouts across screen sizes. Keep form labels, validation errors, result sections, and navigation easy to scan.

Admin boundary

The admin page is intended for token-authenticated viewing and deletion. Do not expose administrative controls as ordinary user actions.

DELIVERABLES

- Profile input template
- Personalized result and feedback template
- Admin user-list template
- Responsive shared stylesheet

ACCEPTANCE CHECKS

- Form fields correspond to backend schema
- Generated and updated content render in the right states
- Error responses are understandable and layout remains usable on smaller screens

Local Deployment & Verification

PROJECT PLAN | SCOPE & OBJECTIVES

PHASE OVERVIEW

Objective

Prepare FitBuddy to run locally and confirm its main interfaces and workflows are reachable.

Included work

Install & configure

Create a virtual environment, install requirements, and select demo mode or configure a Gemini API key.

Start the service

Run Uvicorn from the FitBuddy project directory and open the local home page.

Verify interfaces

Check the home page, FastAPI docs, health endpoint, and admin page as appropriate.

Exercise core behavior

Run the automated project test suite and manually inspect generation, persistence, feedback, and error states.

PHASE OUTPUT

This phase produces a working, reviewable foundation for the next project phase.

Local Deployment & Verification

PROJECT PLAN | IMPLEMENTATION & ACCEPTANCE

Deployment checklist & handoff

Local launch

python -m venv .venv; activate the environment; pip install -r requirements.txt; uvicorn app.main:app --reload.

Useful URLs

http://127.0.0.1:8000 for the app; /docs for API exploration; /health for service status; /view-all-users for the admin screen.

AI modes

The README documents `AI_DEMO_MODE=true` for key-free local evaluation. For real Gemini generation, set `GOOGLE_API_KEY`, set `AI_DEMO_MODE=false`, then restart.

Release readiness

Confirm configuration is not committed with real secrets, inspect expected behavior, and provide clear run instructions for the next developer.

DELIVERABLES

- Repeatable local installation and run steps
- Verified health, UI, and API entry points
- Test and manual verification record
- Configuration notes for demo and real Gemini use

ACCEPTANCE CHECKS

- App starts and key pages are reachable
- Database-backed generation and feedback flow work in selected mode
- pytest completes successfully; any gaps are recorded